Regularizing Groups, Not Parameters

FIM Null-Space Penalties for Structurally Non-Identifiable Parameter Groups

Companion to `notes/fim_l2_regularization_calculations.tex`
Reference implementation: `scripts/fim_l2_constraintL2.R`

1 The core idea

Near a fit p^*, the objective looks like a bowl. If a group of parameters is structurally non-identifiable, that bowl is not round: it is a long, flat valley in some direction and steep in every other. The valley direction is, in general, *not* the axis of any single raw parameter – it is a mix of several. Figure 1 sketches this for two parameters p_1, p_2.

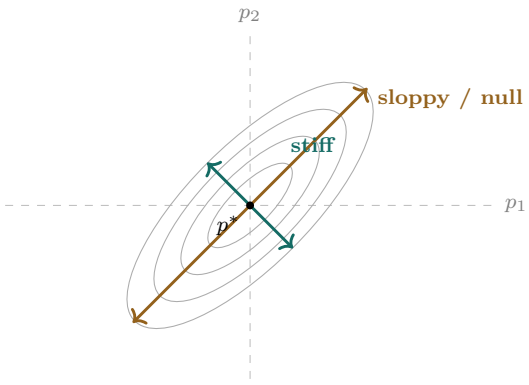


Figure 1: Contours of the objective near a fit p^*. The **amber** and **teal** axes are the valley's own directions, not the raw p_1, p_2 axes (grey, dashed). A penalty built directly from p_1 and p_2 individually cannot avoid mixing both.

The rest of this note builds a penalty aligned with the amber axis and nothing else. First we need a way to *find* that axis numerically – that is what the Fisher Information Matrix (FIM) gives us.

2 Setup: the local quadratic model

Expand the objective around a reference point p^* (typically your best fit so far). To second order, with the first-order term vanishing at a fit,

$$\text{obj}(p) \;\approx\; \text{obj}(p^*) \;+\; \tfrac{1}{2}\,(p-p^*)^\top H\,(p-p^*), \tag{1}$$

where H is the Gauss–Newton Hessian – the same quantity `fim_l2_constraintL2.R` already computes as `o_best$hessian`, i.e. $H = S^\top W S$ with S the sensitivities and W the inverse-variance weights. This *is* the Fisher Information Matrix.

Why expand around a point at all?

This step is not a formality separate from "just take the Hessian" – it is what makes H well defined in the first place. dMod's objective is built from ODE solutions and is generally nonlinear in p; there is no single, global "Hessian of the model" the way there would be for an actually-quadratic objective. What exists is $H = \nabla^2 \text{obj}(p^*)$, the curvature *at one specific point* – evaluate it somewhere else and you get a different matrix, different eigenvectors, possibly a different flat direction.

Two consequences follow directly from this:

- p^* is not just where H happens to live – it is also the point the penalty in Section 4 is centered on, $R(p) = \lambda(p - p^*)^\top P(p - p^*)$. The penalty acts on the *displacement* $p - p^*$, and p^* is the only point that displacement can meaningfully be measured from, since it is exactly where the curvature was measured.
- Everything built from H – the stiff/sloppy split, V_{null}, the projector P – is therefore a *local* description, accurate only near p^* (cf. "Local, not global" in Section 8). In the worked example below, the valley happens to be exactly straight everywhere, because the toy model depends on $u + v$ globally and not just locally, which is why H there is exact rather than a local approximation; for a generic nonlinear model this need not hold, and V_{null} found at p^* should be treated as trustworthy only in a neighborhood of p^*.

Diagonalize it (`eig <- eigen(H, symmetric = TRUE)`):

$$H \;=\; V\,\text{diag}(s_1, \ldots, s_n)\,V^\top. \tag{2}$$

V is orthogonal – its columns $v_1, \ldots, v_n$ are the curvature's own axes, each perpendicular to every other. s_i is how sharply the objective curves along v_i: large s_i means that direction is pinned down tightly by the data (stiff); s_i near zero means the data barely constrains it at all (sloppy) – moving along v_i changes the objective almost not at all, exactly what a structural non-identifiability looks like numerically.

3 Splitting the space

Sort the eigenvalues by size and threshold them – you already do exactly this:

$$\texttt{is_zero}_i \;=\; \mathbb{1}\big[\,|s_i| < 10^{-6} \cdot \max_j |s_j|\,\big]. \tag{3}$$

This splits the eigenbasis into two groups of columns:

- V_{stiff} – eigenvectors with non-negligible curvature: directions the data actually pins down.
- V_{null} – eigenvectors with (numerically) zero curvature: directions the data says nothing about, i.e. the non-identifiable group, expressed in its own natural coordinates rather than yours.

V_{null} is a set of *directions* in parameter space, generally a mix of several raw parameters (the amber axis in Figure 1), *not* a subset of parameter names the way `mu` in `constraintL2(mu, sigma)` is.

4 The null-space penalty

Build the matrix that projects any vector onto V_{null}'s span:

$$P \;=\; V_{\text{null}}\, V_{\text{null}}^{\top}. \tag{4}$$

Because the columns of V_{null} are orthonormal, P is symmetric and *idempotent*, $PP = P$ (projecting twice does nothing a single projection didn't already do). Penalize only the component of p that lands inside that projection:

$$R(p) \;=\; \lambda\,(p-p^*)^{\top} P\,(p-p^*) \;=\; \lambda\,\big\|V_{\text{null}}^{\top}(p-p^*)\big\|^2. \tag{5}$$

The right-hand form is the more intuitive one: $z = V_{\text{null}}^{\top}(p-p^*)$ is just $p-p^*$ re-expressed in the sloppy eigenbasis, and $R = \lambda\|z\|^2$ is an ordinary, isotropic L2 penalty – but applied to z, not to p. It is the same trick as regularizing "only some parameters," except the coordinates being regularized are a rotated/projected combination discovered from the FIM, not a subset of names picked by hand.

Value, gradient, Hessian

Because R is exactly quadratic, all three are closed-form and exact – no chain rule, no `deriv2` needed:

$$\nabla R \;=\; 2\lambda P\,(p-p^*), \qquad \nabla^2 R \;=\; 2\lambda P \quad \text{(constant, independent of } p\text{)}. \tag{6}$$

Why the identified combination is untouched

Take any displacement δ that lies purely in the stiff subspace, $\delta = V_{\text{stiff}}\, c$ for some coefficients c. Since V's columns are mutually orthogonal by construction, $V_{\text{null}}^{\top} V_{\text{stiff}} = 0$, so

$$P\delta \;=\; V_{\text{null}}\big(V_{\text{null}}^{\top} V_{\text{stiff}}\big)c \;=\; 0 \qquad \text{exactly, not approximately – for any } c. \tag{7}$$

R and its gradient are *identically* zero for any move confined to the identified directions. This is the property a per-parameter `constraintL2` prior cannot offer: unless a sloppy eigenvector happens to line up with one raw parameter axis, no choice of diagonal σ_i can null out a direction exactly – it can only make the leakage smaller.

5 Worked example

Take the multiplicative symmetry discussed earlier: a model where only the product $k_1 k_2$ is identifiable, not k_1 and k_2 separately. Working in dMod's usual log-parametrization, $u = \log k_1$, $v = \log k_2$, the data only ever sees $u+v$. Near the fit, with data-curvature constant c along $u+v$, the objective's curvature is

$$H \;=\; c\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \qquad c = 4. \tag{8}$$

This H is exactly singular – not just small in one direction, genuinely zero – which makes the eigendecomposition easy to do by hand.

So $V_{\text{null}} = (1,-1)/\sqrt{2}$, and the projector is

$$P \;=\; V_{\text{null}} V_{\text{null}}^{\top} \;=\; \begin{pmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{pmatrix}. \tag{9}$$

eigenvalue s	eigenvector v	reading
8	$(1, 1)/\sqrt{2}$	**stiff** – the identified direction, $u + v$
0	$(1, -1)/\sqrt{2}$	**sloppy** – the null direction, $u - v$

Table 1: Eigendecomposition of H.

Writing $\delta = (\delta_u, \delta_v) = (u - u^*, v - v^*)$, the penalty reduces to a single term:

$$R(u, v) \;=\; \tfrac{1}{2}\lambda\,(\delta_u - \delta_v)^2, \tag{10}$$

which depends *only* on the gap $\delta_u - \delta_v$ – i.e. only on how far the current gauge has drifted from the reference gauge – never on $\delta_u + \delta_v$, the quantity the data actually fits.

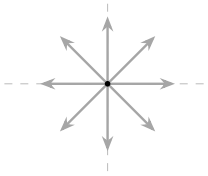
6 Naive vs. projector, side by side

Compare the gradients of the two penalties directly. "Naive" is `constraintL2(mu = c(u=u*, v=v*), sigma = c(`σ, σ`))`, i.e. $R_{\text{naive}} = (\delta_u^2 + \delta_v^2)/\sigma^2$. The number that matters is $\partial R/\partial u + \partial R/\partial v$ – the net push along the stiff direction $(1, 1)$, i.e. against the identified combination:

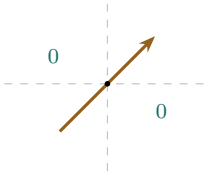
	$\partial R/\partial u$	$\partial R/\partial v$	sum (pull on $u + v$)
naive, diagonal	$2\delta_u/\sigma^2$	$2\delta_v/\sigma^2$	$2(\delta_u + \delta_v)/\sigma^2$ – **nonzero** unless $\delta_u = -\delta_v$
projector	$\lambda(\delta_u - \delta_v)$	$-\lambda(\delta_u - \delta_v)$	0 – **always, for every** δ

Table 2: Net pull on the identified combination $u + v$.

The naive penalty's sum is zero only on the razor's edge where $\delta_u = -\delta_v$ exactly – everywhere else it quietly drags the fit's best-supported quantity, $u + v$, toward $u^* + v^*$ too, competing with the data even though the data already had an opinion there. The projector's sum is zero identically – it has no opinion about $u + v$ at all, by construction.



Naive: equal pull regardless of what the data already knows.



Projector: pull confined to V_{null}; the stiff direction reads exactly 0.

Figure 2: Naive isotropic pull (left) vs. null-space projector pull (right).

7 From formula to code

`constraintL2()` cannot express this directly – it only builds a diagonal penalty in the raw parameter names (`R/objClass.R`, around line 433). P is generally dense, so the penalty needs its own small `objfn`, built the same way `constraintL2` is: return an `objlist` with exact value, gradient and Hessian, and let `+` fold it into the rest of the objective.

```
# H, eig, s, is_zero, best already computed in fim_l2_constraintL2.R, Part 8
V_null <- eig$vectors[, is_zero, drop = FALSE] # n x m, m = # non-identifiable
    directions
P <- V_null %*% t(V_null) # n x n projector, symmetric, P %*% P == P

groupL2 <- function(P, p0, lambda = 0.1) {
  force(P); force(p0); force(lambda)
  function(..., fixed = NULL, deriv = TRUE, deriv2 = FALSE,
           conditions = NULL, env = NULL) {
    p <- list(...)[[1]]
    delta <- p[names(p0)] - p0
    Pd <- as.numeric(P %*% delta)

    value <- lambda * sum(delta * Pd) # R(p) = lambda * delta' P delta
    grad <- setNames(2 * lambda * Pd, names(p0)) # exact, no deriv2 needed
    hess <- 2 * lambda * P
    dimnames(hess) <- list(names(p0), names(p0))

    out <- objlist(value = value, gradient = grad, hessian = hess)
    attr(out, "prior") <- value
    attr(out, "env") <- env
    out
  }
}

prior_group <- groupL2(P, best, lambda = 0.1)
class(prior_group) <- c("objfn", "fn")
attr(prior_group, "parameters") <- names(best)

obj_group <- obj + prior_group # "+.fn" sums value / gradient / hessian, as usual
```

If `best` sits behind a `trafo`/`P()` reparameterization, apply this at the same (outer) parameter level H was computed at – the same level `constraintL2` already operates on – so no extra chain rule is required.

8 Practical notes

Local, not global. H is a snapshot at p^*. If the true symmetry is curved in your raw parameters (it usually is not, once you are already in log-space – multiplicative symmetries become straight lines exactly like the worked example), V_{null} is only accurate near p^*; re-evaluate H if the fit moves far from where it was computed.

Relation to your `sigma_fim` construction. The existing anisotropic recipe uses a *soft*, reciprocal weight $w_i = \texttt{strength}/(s_i + \texttt{eps_reg})$ for every eigendirection, then keeps only the diagonal of $V\,\mathrm{diag}(w)\,V^\top$ – every direction gets *some* pull, and cross-parameter structure is discarded when collapsing to a diagonal σ. The projector above is the same construction pushed to its limit: a *hard* 0/1 weight (`is_zero`) instead of a soft reciprocal one, and the full dense P kept intact instead of flattened to a diagonal. The result is a strict either/or: a direction is either fully exempt or fully regularized, and the "how strongly" knob (λ) is decoupled from "which

directions" (the mask) – whereas today those two questions are answered by the same number, `strength / (s_i + eps_reg)`.

If you know the symmetry analytically. When `symmetryDetection()` (`R/symmetryDetection.R`) can name the symmetry generator in closed form, that is strictly better than any numerical V_{null}: reparametrize via `repar()` into (identified combination, gauge coordinate) directly, and `constraintL2()` the gauge coordinate alone. $z = V_{\text{null}}^{\top}(p - p^*)$ above is exactly a numerical stand-in for that gauge coordinate, useful precisely when deriving it by hand is not practical.